

PARCS-Agent: Enabling AI Agents to Exploit Distributed Parallel Computing via the Model Context Protocol

Oleksii Bohusevych ¹ 

¹ Faculty of Computer Science and Cybernetics, Taras Shevchenko National University of Kyiv, 01601 Kyiv, Ukraine; oleksii.bohusevych@knu.ua

* Correspondence: oleksii.bohusevych@knu.ua

1 Abstract

Large language model (LLM) agents have demonstrated impressive capabilities in tool use, code generation, and scientific reasoning, yet they remain bound by the throughput of a single execution thread. Computationally intensive tasks—Monte Carlo simulation, combinatorial optimisation, hyperparameter search, large-scale graph analysis—either time out or require severe undersampling when handled sequentially. We present PARCS-Agent, a system that exposes a cloud-scale distributed computing cluster to an AI agent through three Model Context Protocol (MCP) tools: `get_cluster_info`, `create_session`, and `run_layer`. The agent submits arbitrary C# computation logic; the platform compiles it with Roslyn at runtime, fans it out across up to 21 Kubernetes worker pods via Google Cloud Pub/Sub and KEDA autoscaling, and returns aggregated results. We introduce the PARCS-Agent Benchmark, a suite of 15 tasks spanning quantitative finance, bioinformatics, epidemiology, machine learning, cryptography, and atmospheric science, designed so that the sequential wall-clock cost substantially exceeds practical agent time budgets. Experiments show that PARCS-enabled agents achieve mean speedups of 15–18× over sequential baselines while delivering statistically equivalent or superior solution quality. PARCS-Agent is, to our knowledge, the first system to use MCP as the interface between an AI agent and a live distributed HPC cluster, and the first benchmark to measure *speedup ratio* as a first-class agent capability metric.

Keywords: AI agents; Model Context Protocol; distributed computing; parallel computation; LLM tool use; benchmarking; Kubernetes; high-performance computing

1. Introduction

2. Related Work

2.1. Tool-Augmented Language Model Agents

Two main directions can be distinguished in the literature on augmenting language models with external tools. The first, introduced by Toolformer by Schick et al. [1], showed that a model is capable of learning, without supervised examples, when and how to call external APIs such as calculators, search engines, or translation services. The second, ReAct by Yao et al. [2], proposed a tighter integration of reasoning and action: the model produces a chain-of-thought trace, takes an action, receives an observation, and repeats. Both works established the agent-tool loop as a general paradigm, and many subsequent systems build directly on top of them.

The scope of tool use has since expanded considerably. ToolLLM by Qin et al. [3] curated a collection of 16,000 real-world REST endpoints and trained a model to select and invoke them using a tree-based search strategy. Gorilla by Patil et al. [4] addressed the specific problem of hallucinated API signatures when working with ML framework APIs, showing that retrieval-augmented generation can mitigate this. SciAgent by Ma et al. [5] brought scientific computing tools into the picture, assembling

over 30,000 tool-annotated examples spanning physics, chemistry, mathematics, and finance, alongside a dedicated benchmark. Among the broader surveys, the one by Mialon et al. [6] is notable for placing all of the above within a taxonomy of memory, reasoning, and action, and for noting that large-scale computation as a tool type remains underexplored.

In all of the aforementioned systems, a tool call is stateless and local: it takes inputs, returns a result, and exits. PARCS-Agent departs from this model, as its `run_layer` tool submits user-written C# code to a running Kubernetes cluster, fans it out across a pool of workers via a Pub/Sub queue, and aggregates their output. Hence, a single tool invocation may correspond to dozens of parallel processes rather than a single function call.

LLMCompiler by Kim et al. [7] is worth singling out, as it does introduce parallelism into tool use. It analyses the dependency structure of a query, builds a DAG of tool calls, and dispatches independent branches concurrently, achieving up to $3.7\times$ latency reduction on multi-hop tasks. The said parallelism, however, applies to independent calls to existing tools, not to distributed execution of user-supplied code. CodeAct by Wang et al. [8] proposes Python as the agent’s action space and runs it in a stateful interpreter that persists across turns. At the same time, execution remains single-threaded and local, making it unsuitable for computationally intensive tasks.

2.2. LLMs for High-Performance and Parallel Computing

On the one hand, there is a growing body of work examining whether LLMs can produce correct parallel code. ParEval by Nichols et al. [9] comprises 420 problems drawn from numerical computing, graph algorithms, and Monte Carlo methods, and evaluates generated code targeting MPI, OpenMP, CUDA, and Kokkos. The findings indicate that state-of-the-art models generate significantly less correct parallel code than serial code for the same problems, with correctness declining notably as task complexity grows. HPC-Coder-V2 [10] brought improvements in this direction by fine-tuning on a corpus of 120,000 parallel code samples, reporting gains on MPI and OpenMP generation in particular. Separately, recent frontier models were evaluated on classical HPC kernels such as matrix multiplication and conjugate gradient, with results indicating that non-trivial memory access patterns remain problematic [11].

On the other hand, PARCS-Agent sidesteps the said code generation problem altogether. The agent writes only per-worker logic as a plain C# class implementing a fixed interface; the platform compiles it via Roslyn, distributes the resulting binary to worker pods, and handles all coordination. As such, the agent is never required to reason about parallelism in code, and the gap identified by ParEval becomes irrelevant when the parallel infrastructure is a managed service.

He et al. [12] applied LLMs to a different HPC problem, namely the automated optimisation of the task-to-processor mapper in the Legion distributed runtime. The Legion mapper DSL served as the target representation, and the system proposed mapping strategies that previously required days of expert tuning. It is worth mentioning, however, that the said work presupposes an already-parallelised application; the problem it addresses is performance tuning rather than enabling new computations to be executed in parallel from scratch.

2.3. Agent Benchmarks for Computational Tasks

Several benchmarks have been proposed in recent years to measure agent performance on complex, multi-step tasks. GAIA [13] comprises 466 questions requiring a combination of web browsing, file parsing, and multi-modal reasoning; the hardest third of questions remains largely unsolved. SWE-bench [14] evaluates whether agents can resolve real GitHub issues by producing patches that pass the repository’s test suite, with top systems solving roughly half of the verified subset. ScienceAgentBench [15] consists of 102 tasks drawn from 44 published papers, spanning bioinformatics, chemistry, and geographical information science, with the best agent completing around one-third autonomously. TheAgentCompany [16] takes a different angle, benchmarking agents on workplace tasks in a simulated company environment.

All of the above benchmarks share a common assumption: tasks are designed to be solvable by a single-threaded agent within a reasonable wall-clock budget, and compute time is not a measured variable. The PARCS-Agent Benchmark is constructed around the opposite premise. Each of the 15 tasks carries a sequential cost that is impractical to complete within a reasonable budget, and the central result is the ratio of wall-clock time between a PARCS-enabled agent and a sequential baseline. Hence, speedup ratio serves as a primary evaluation criterion, which, to the best of the author's knowledge, has not been used in agent benchmarking before.

2.4. The Model Context Protocol

The Model Context Protocol (MCP) was introduced by Anthropic in 2024 as a standard interface for connecting AI agents to external tools, data, and prompts via JSON-RPC over server-sent events [17]. Hou et al. [18] surveyed the MCP ecosystem and identified security concerns comprising prompt injection, tool poisoning, and privilege escalation; notably, no prior work on exposing distributed compute infrastructure via MCP was found. A separate analysis of published MCP tool descriptions found that over half lacked adequate documentation of purpose or parameter constraints [19]. The said findings motivated a deliberate effort in the design of the PARCS-Agent tools, each of which specifies parameter bounds, error conditions, and expected return values explicitly.

As far as the author is aware, PARCS-Agent is the first system to expose a live parallel computing cluster as a set of MCP tools, and the first benchmark to evaluate agent performance specifically on tasks where access to such a cluster is the deciding factor.

3. System Architecture

4. PARCS-Agent Benchmark

5. Experiments

6. Discussion

7. Conclusions

Author Contributions: Conceptualization, O.B.; methodology, O.B.; software, O.B.; validation, O.B.; formal analysis, O.B.; investigation, O.B.; data curation, O.B.; writing - original draft preparation, O.B. The author has read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The benchmark dataset is publicly available at <https://huggingface.co/datasets/parcs-benchmark/parcs-agent-benchmark>. The platform source code is available at <https://github.com/alexbohuslav/parcs7>.

Acknowledgments: In this section you can acknowledge any support given which is not covered by the author contribution or funding sections. This may include administrative and technical support, or donations in kind (e.g., materials used for experiments).

Conflicts of Interest: Declare conflicts of interest or state "The authors declare no conflicts of interest." Authors must identify and declare any personal circumstances or interest that may be perceived as inappropriately influencing the representation or interpretation of reported research results. Any role of the funders in the design of the study; in the collection, analyses or interpretation of data; in the writing of the manuscript; or in the decision to publish the results must be declared in this section. If there is no role, please state "The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results".

Abbreviations

The following abbreviations are used in this manuscript:

MCP	Model Context Protocol
LLM	Large Language Model
HPC	High-Performance Computing
API	Application Programming Interface
KEDA	Kubernetes Event-Driven Autoscaling
GKE	Google Kubernetes Engine
SA	Simulated Annealing
VaR	Value-at-Risk
CVaR	Conditional Value-at-Risk

Appendix A

Appendix A.1

The appendix is an optional section that can contain details and data supplemental to the main text—for example, explanations of experimental details that would disrupt the flow of the main text but nonetheless remain crucial to understanding and reproducing the research shown; figures of replicates for experiments of which representative data are shown in the main text can be added here if brief, or as Supplementary Data. Mathematical proofs of results not central to the paper can be added as an appendix.

Table A1. This is a table caption.

Title 1	Title 2	Title 3
Entry 1	Data	Data
Entry 2	Data	Data

Appendix B

All appendix sections must be cited in the main text. In the appendices, Figures, Tables, etc. should be labeled, starting with “A”—e.g., Figure A1, Figure A2, etc.

References

- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Zettlemoyer, L.; Cancedda, N.; Scialom, T. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*; Curran Associates: Red Hook, NY, USA, 2023.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*; OpenReview.net, 2023.
- Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*; OpenReview.net, 2024.
- Patil, S.G.; Zhang, T.; Wang, X.; Gonzalez, J.E. Gorilla: Large Language Model Connected with Massive APIs. *arXiv* **2023**, arXiv:2305.15334.
- Ma, Z.; Gou, Z.; Hao, J.; Xu, K.; Hu, Z.; Yin, C.; Zheng, H.; Huang, M.; Deng, Y. SciAgent: Tool-Augmented Language Models for Scientific Reasoning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2024.
- Mialon, G.; Dessì, R.; Lomeli, M.; Nalmpantis, C.; Pasunuru, R.; Raileanu, R.; Rozière, B.; Schick, T.; Dwivedi-Yu, J.; Celikyilmaz, A.; et al. Augmented Language Models: A Survey. *Trans. Mach. Learn. Res.* **2023**.
- Kim, S.; Moon, S.; Tabrizi, R.; Lee, N.; Mahoney, M.W.; Keutzer, K.; Gholami, A. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*; PMLR: Honolulu, HI, USA, 2024; Vol. 235, pp. 24167–24190.

8. Wang, X.; Chen, Y.; Yuan, L.; Zhang, Y.; Li, Y.; Peng, H.; Ji, H. Executable Code Actions Elicit Better LLM Agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*; PMLR: Honolulu, HI, USA, 2024; Vol. 235, pp. 50908–50933.
9. Nichols, D.; Davis, J.H.; Xie, Z.; Rajaram, A.; Bhatele, A. Can Large Language Models Write Parallel Code? In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing (HPDC 2024)*; ACM: New York, NY, USA, 2024; pp. 117–130.
10. Nichols, D.; Davis, J.H.; Bhatele, A. HPC-Coder-V2: Studying Code LLMs Across Low-Resource Parallel Languages. *arXiv* **2024**, arXiv:2412.15178.
11. Xu, C.; Li, M.; Xu, X.; Qin, X.; Chen, P. LLM & HPC: Benchmarking DeepSeek’s Performance in High-Performance Computing Tasks. In *Proceedings of ISC High Performance 2025*; Springer: Cham, Switzerland, 2025.
12. He, W.; Shen, Z.; Nie, A.; Luo, C.; Nus, T.; Aiken, A.; Zaharia, M. Improving Parallel Program Performance with LLM Optimizers via Agent-System Interfaces. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*; PMLR, 2025.
13. Mialon, G.; Fourrier, C.; Swift, C.; Wolf, T.; LeCun, Y.; Scialom, T. GAIA: A Benchmark for General AI Assistants. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*; OpenReview.net, 2024.
14. Jiménez, C.E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; Narasimhan, K. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*; OpenReview.net, 2024.
15. Chen, Z.; Shi, Z.; Wang, S.; et al. ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery. In *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*; OpenReview.net, 2025.
16. Xu, F.F.; Ye, Y.; Jain, A.; Du, Y.; et al. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. *arXiv* **2024**, arXiv:2412.14161.
17. Anthropic. Model Context Protocol. Available online: <https://modelcontextprotocol.io> (accessed on 1 May 2026).
18. Hou, X.; Zhao, Y.; Wang, S.; Wang, H. Model Context Protocol (MCP): Landscape, Security Threats and Future Research Directions. *arXiv* **2025**, arXiv:2503.23278.
19. Luo, L.; et al. Model Context Protocol (MCP) Tool Descriptions Are Smelly! *arXiv* **2025**, arXiv:2602.14878.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.